ÉPREUVE E2 · ENTRAÎNEMENT

VéloVille

Une plateforme pour fiabiliser et valoriser les données d'un service de vélos en libre-service.

API FastAPI · PostgreSQL · Interface Streamlit · Version 1.0.3



01 · BESOIN

Du CSV brut à une information exploitable

Contexte

Les équipes doivent consulter les stations, suivre les trajets et traiter les incidents à partir d'exports hétérogènes.

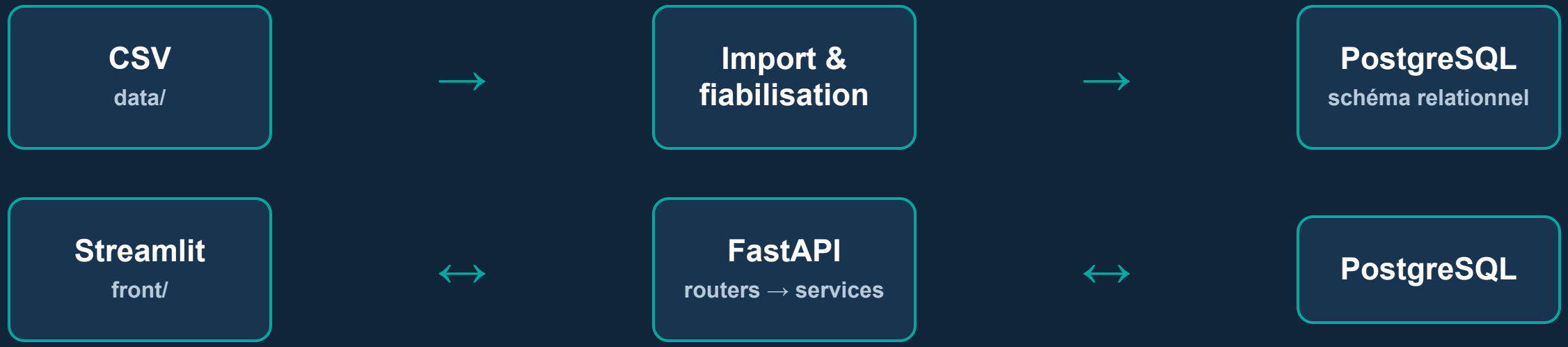
- Plusieurs fichiers et formats d'entrée
- Références parfois invalides ou incohérentes
- Accès aux données selon le rôle utilisateur

Objectifs livrés

- Nettoyer et tracer les anomalies sans perte silencieuse
- Fournir une API sécurisée et paginée
- Présenter les données dans une interface tabulaire sobre
- Documenter l'installation, les tests et l'exploitation

02 · ARCHITECTURE

Des responsabilités séparées



Les routeurs exposent le contrat HTTP ; les services portent les requêtes et traitements métier.

Technologies : Python · SQLAlchemy · JWT · bcrypt

Audit des trois exports fournis

120

Stations

Aucune anomalie bloquante constatée dans le jeu fourni.

2 002

Trajets importables

Sur 2 030 lignes : 16 rejets et 12 doublons stricts écartés.

177

Incidents importables

Sur 180 lignes : 3 rejets pour dates de résolution incohérentes.

> Les avertissements de normalisation annotent des lignes importées ; ils ne sont pas comptés comme rejets.

Exemples : 1 186 pseudonymes et 6 identifiants vélo normalisés ; 24 types d'incident harmonisés.

Un schéma relationnel contrôlé

Entités

- **station** : capacité et coordonnées vérifiées
- **trajet** : départ obligatoire, arrivée nullable
- **incident** : cible, statut et résolution cohérents
- **utilisateur** : rôle et hachage de mot de passe

Garanties principales

- Clés primaires et références étrangères
- Durée positive et coordonnées dans les bornes
- Règle pseudonyme / type d'usager respectée
- Date de résolution cohérente avec le statut

Les règles sources sont appliquées avant insertion ; les contraintes critiques sont aussi déclarées dans PostgreSQL et l'ORM.

Traçable et rejouable

1
Valider

Normaliser formats, identifiants et références ; rejeter les valeurs incompatibles avec un motif.

2
Charger

Importer les stations avant les trajets et incidents ; ne pas dupliquer les données acceptées.

3
Auditer

Générer un journal à six colonnes : source, identifiant, colonne, valeur, motif, décision.

> Correction vérifiée : au second passage, une ligne déjà en base n'est plus comptée comme doublon du CSV.

Un test d'intégration SQLite rejoue l'import deux fois et vérifie compteurs, journal et volumes inchangés.

Accès maîtrisés par rôle

Authentification

- Mots de passe vérifiés avec bcrypt
- Jetons JWT signés, avec expiration
- Réponses 401 uniformes pour identifiants invalides
- Routes protégées par dépendances d'authentification

Autorisations

- **Exploitant** : lecture et changement du statut incident
- **Direction** : consultation des données et indicateurs
- Pagination avec bornes serveur
- Transitions d'incident validées dans les services

Trois agrégations, une vue lisible

Activité quotidienne

Nombre de trajets par jour, filtré par période.

Stations actives

Classement des départs et arrivées par station.

Incidents

Répartition par statut et délai moyen de résolution.

La page effectue exactement **trois appels API** ; le total de trajets est la somme des résultats quotidiens.

Filtres appliqués avec Actualiser ; états vides explicites, sans graphique ni rafraîchissement automatique.

Tests automatisés et état de recette

57

Tests réussis

Tests unitaires, import rejoué et tests API/front.

93 %

Couverture services

Mesurée par pytest-cov sur les services.

10 / 10

Cas de recette OK

API, imports PostgreSQL, indicateurs et parcours front vérifiés.

> La restauration d'une sauvegarde reste à valider séparément sur un environnement dédié.

Résultats détaillés : docs/recette/resultats-recette.md

Une base livrable, des contrôles à terminer

Maintenance prévue

- Sauvegarde PostgreSQL horodatée avec pg_dump
- Procédure de déploiement et de restauration documentée
- Variables locales séparées dans .env
- Historique versionné sans secrets

Prochaines actions

- Automatiser les sauvegardes et surveiller leur résultat
- Tester pg_restore sur une base dédiée
- Formaliser la reprise après incident
- Conserver les preuves et faire évoluer la plateforme

Merci — questions ?